

STASIS

Save System

Complete Product Documentation

Version 1.0.0

RangaDigital

© 2026 RangaDigital. All rights reserved.

Version and Legal Information

Product	Stasis Save System
Version	1.0.0
Publisher	RangaDigital
Unity Compatibility	Unity 6 (6000.x) and later; requires .NET Standard 2.1
Package ID	com.rangadigital.stasis
Document Date	April 06, 2026
Document Revision	1.0
License	Commercial. Distribution, resale, and redistribution of source code are prohibited without explicit written permission from RangaDigital.

Copyright Notice

Copyright © 2025 RangaDigital. All rights reserved. This product, its source code, compiled binaries, documentation, and all associated assets are the exclusive intellectual property of RangaDigital. Unauthorised copying, modification, distribution, or use of this product or any portion thereof is strictly prohibited and may result in civil and criminal liability under applicable copyright law.

Disclaimer of Warranties

This product is provided on an 'as-is' basis. RangaDigital makes no warranties, express or implied, including but not limited to warranties of merchantability, fitness for a particular purpose, or non-infringement. In no event shall RangaDigital be liable for any direct, indirect, incidental, special, exemplary, or consequential damages arising out of the use or inability to use this product, even if advised of the possibility of such damages.

Third-Party Acknowledgements

This product integrates the following third-party libraries under their respective licences:

- **System.Text.Json** — Part of the .NET Standard library. Used for JSON serialisation. Included under the MIT Licence.
- **K4os.Compression.LZ4** — LZ4 fast compression library. Used for high-speed payload compression. Included under the BSD Licence.
- **ZstdSharp** — Managed Zstandard compression wrapper. Used for high-ratio compression. Included under the MIT Licence.
- **Unity.Cinemachine (optional)** — Unity Technologies Cinemachine package. Used for optional camera state persistence. Subject to the Unity Package Distribution Licence.
- **Unity.Timeline (optional)** — Unity Technologies Timeline package. Used for optional timeline state persistence. Subject to the Unity Package Distribution Licence.

- **Unity.VisualEffectGraph (optional)** — Unity Technologies VFX Graph package. Used for optional VFX state persistence. Subject to the Unity Package Distribution Licence.

1. Introduction and Overview

What Is This Product?

Stasis is a professional, production-grade save system for Unity 6 games. It provides a complete, end-to-end solution for persisting game state — from a single integer value to complex spawned object hierarchies — with zero compromise on reliability, performance, or security. At its core, Stasis serialises an in-memory document model into any of four binary or text formats, applies optional compression and authenticated encryption, and writes the result to disk atomically so that a mid-write power failure can never corrupt a save file.

This product was designed from first principles to address the persistent shortcomings of every other save-system solution on the Unity Asset Store: opaque data formats that cannot be inspected or migrated, save corruption on crash, no encryption, no versioning, and single-threaded blocking I/O that causes frame stutter. Stasis solves every one of these problems in a clean, inspector-driven API that requires no boilerplate.

Problems This Product Solves

Save Corruption

Every write is performed atomically: data is written to a temporary file, then the original is replaced in a single OS-level rename operation. A crash mid-write never leaves a partially-written save. Automatic multi-generation backups (.bak, .bak.1, .bak.2) ensure that recovery is always possible.

Performance and Frame Stutter

All I/O is fully asynchronous. Serialisation, compression, encryption, and disk writes never block the Unity main thread. A dedicated async pipeline handles the complete save/load lifecycle without a single frame drop.

Security and Integrity

Payloads can be protected with AES-256-GCM authenticated encryption (IL2CPP builds) or AES-256-CBC with a random IV (Mono builds). GCM mode provides authenticated encryption, meaning any tampering with the save file is detected and rejected at load time. An optional HMAC-SHA256 signing layer adds a second integrity check independent of encryption.

Format Inflexibility

Stasis supports four serialiser profiles selectable at runtime: JSON (human-readable, ideal for development), Binary Standard (compact, fully-typed), Binary Fast / MemoryPack (maximum throughput via zero-copy deserialisation), and a Development JSON Override that transparently substitutes JSON in debug builds without changing production data.

Migration and Versioning

A structured migration pipeline allows save files produced by earlier versions of the game to be upgraded to the current schema automatically at load time, with per-field defaults, rename mappings, and custom migration handlers.

Spawn and Scene State

Stasis natively serialises spawned GameObjects — objects that did not exist in the original scene at edit time — tracking their prefab address, position, rotation, and any registered Stasis-aware components.

Platform Fragmentation

Platform Profiles allow different compression, encryption, and I/O strategies to be configured per target platform at edit time, with runtime switching handled automatically by the system.

Core Value Proposition

Stasis delivers the equivalent of a game-studio-grade save-system implementation — one that would typically take a dedicated engineer months to build and harden — as a plug-and-play Unity package. It eliminates an entire category of production risk from any Unity game project: save corruption, security vulnerabilities, performance regressions on slow storage devices, and the maintenance burden of hand-rolled serialisation code.

Target Audience

This product is intended for professional Unity game developers, studios, and independent developers who require a robust, production-ready save system and cannot afford to build one from scratch or accept the compromises inherent in simpler alternatives. Suitable audiences include:

- Indie developers building their first commercial title who need production-quality infrastructure without the engineering overhead.
- Mid-sized studios replacing a fragile hand-rolled save system before shipping.
- Technical leads evaluating save-system packages for a new project with security or compliance requirements.
- Developers targeting multiple platforms (PC, console, mobile, WebGL) who need a unified save API with per-platform configuration.

Prerequisites

Unity Version	Unity 6 (6000.x) or later. Unity 2022 LTS is supported with minor limitations (no AesGcm in Mono editor).
Scripting Backend	Mono or IL2CPP. AES-GCM authenticated encryption requires IL2CPP player builds; Mono builds fall back to AES-CBC.
API Compatibility	.NET Standard 2.1 (the Unity default). Do not switch to .NET Framework.
C# Knowledge	Intermediate. Familiarity with async/await, ScriptableObjects, and the Unity inspector is assumed.
Optional Packages	com.unity.timeline, com.unity.cinemachine, com.unity.visualeffectgraph — required only if the corresponding Stasis extension is enabled.
Addressables	Optional. Required only if using Addressable-based prefab spawning. The Stasis core is Addressables-agnostic.

2. Quick Start Guide

This section guides you from a fresh Unity project to a working save-and-load implementation in under ten minutes. Every step is self-contained.

Step 1 — Install the Package

This product is distributed as a Unity Package Manager (UPM) package. To install it:

1. Open your Unity 6 project.
2. Open the Package Manager window (Window → Package Manager).
3. Click the + button in the top-left corner and select Add package from disk.
4. Navigate to the directory containing the package's package.json file (com.rangadigital.stasis) and select it.
5. Unity will resolve the package and its dependencies automatically. Wait for the import to complete.

■ If you have received this product as a .unitypackage file, double-click it to import it directly. The package structure will be placed under Assets/RangaDigital/Stasis.

Step 2 — Create a Stasis Settings Asset

All runtime configuration is stored in a StasisSettings ScriptableObject asset. This product will remind you to create one if it is missing, but the recommended approach is to create it explicitly:

- In the Unity menu bar, navigate to Window → RangaDigital → Stasis → Setup.
- The Stasis Setup window opens. Click Create StasisSettings Asset.
- Save the asset to Assets/Settings/StasisSettings.asset (or any location inside Assets).
- The asset is now linked to the Stasis runtime. You can inspect and modify all settings from the Inspector.

Step 3 — Add the StasisManager to Your Scene

The StasisManager is a singleton MonoBehaviour that coordinates all save and load operations. Add it to your persistent (DontDestroyOnLoad) scene:

- Create an empty GameObject in your persistent scene and name it StasisManager.
- Attach the StasisManager component (Add Component → RangaDigital → Stasis → StasisManager).
- In the Inspector, assign your StasisSettings asset to the Settings field.
- Ensure the GameObject is part of a scene that is loaded for the entire game session.

■ StasisManager enforces a singleton pattern. If a second instance is created, it destroys itself automatically. You do not need to call DontDestroyOnLoad manually; the component does this in its Awake method.

Step 4 — Configure Basic Settings

With the StasisSettings asset selected in the Inspector, configure the minimum required settings for a working implementation:

Serializer Profile	Set to BinaryFast for best performance, or Json for human-readable save files during development.
Encryption Enabled	Leave unchecked for this quick-start. Encryption requires a key to be set before enabling.
Compression Enabled	Check this box and select LZ4 for fast compression with minimal CPU overhead.
Save Slot Count	Set to the number of save slots your game requires (e.g. 3 for a three-slot RPG, 1 for an autosave-only game).
Storage Path Override	Leave blank. Stasis will use Application.persistentDataPath automatically.

Step 5 — Save and Load Your First Document

The following script demonstrates a minimal save-and-load cycle. Attach it to any `GameObject` in your scene:

C# — MyFirstSave.cs

[illegible]

Run the scene. The console will print 'Save complete.' followed by the loaded values. The save file is written to `Application.persistentDataPath` with the extension `.stasis`.

■ You can find the exact path to the save file in the Stasis Setup window under Window → RangaDigital → Stasis → Setup → Diagnostics, or by calling `StasisManager.Instance.GetSavePath("SaveSlot_0")` at runtime.

3. Comprehensive Feature Breakdown

This section covers every feature of this product in depth, explaining what each feature does, why it exists, and how to use it.

3.1 Serialiser Profiles

Stasis supports four serialiser profiles. The active profile is selected in `StasisSettings` and determines the on-disk format for all save files.

Json

Human-readable UTF-8 JSON. Ideal for development builds, modding support, or games where save-file transparency is a design goal. Largest file size; slowest serialisation. Fully interoperable with any JSON tool.

BinaryStandard

Compact binary format with a full type-tagged layout. Smaller than JSON by roughly 60–70%. Suitable for all production targets. Fully backward-compatible across versions.

BinaryFast

MemoryPack-accelerated binary format using a chunked layout with a table of contents. Enables partial deserialisation (loading a single chunk without reading the entire file). Highest throughput. Requires IL2CPP link.xml preservation for AOT builds.

DevelopmentJsonOverride

A runtime flag (not a separate profile) that substitutes the JSON serialiser transparently in Development player builds, regardless of the configured profile. Allows inspection of save files during QA without changing production settings. The build validator blocks a Release build if this flag is enabled.

How to select a profile:

- Open the `StasisSettings` asset in the Inspector.
- Under the Serialization section, locate the Serializer Profile dropdown.
- Select the desired profile. The change takes effect immediately for all subsequent save operations.
- For the `BinaryFast` profile, open Window → RangaDigital → Stasis → Setup and click Generate Link XML to create the IL2CPP preservation file.

3.2 Encryption

Stasis provides payload-level encryption using industry-standard AES-256. Encryption is applied after serialisation and before compression, ensuring that the ciphertext is always a well-formed binary blob regardless of the serialiser profile.

Encryption Modes

AuthenticatedAesGcm — AES-256-GCM with a randomly-generated 96-bit nonce per save. Provides both confidentiality and authentication: any tampering with the ciphertext is detected and the load is rejected. Available on IL2CPP player builds only. Recommended for all production builds.

LegacyAesCbc — AES-256-CBC with a randomly-generated 128-bit IV per save. Provides confidentiality but no integrity authentication. Used automatically as a fallback on Mono builds where `AesGcm` is

unavailable. Not recommended for new projects; included for compatibility with legacy save files.

Configuring Encryption at Design Time

- In StasisSettings, check the Encryption Enabled checkbox.
- Select the encryption mode from the Encryption Mode dropdown.
- In the Encryption Key field, paste a Base64-encoded 32-byte (256-bit) key. Click Generate New Key to create one automatically.
- The key is stored in the StasisSettings asset. For production builds, rotate the key using the Key Rotation window (Window → RangaDigital → Stasis → Key Rotation) before shipping.

Setting the Encryption Key at Runtime

To avoid shipping the encryption key in the asset bundle (recommended for high-security applications), set the key at runtime before the first save or load:

C# — Set encryption key at runtime

```
// Retrieve the key from your secure delivery mechanism
// (e.g., platform keystore, server handshake, or player-derived KDF)
string base64Key = SecureKeyProvider.GetKey();
StasisManager.Instance.Settings.SetRuntimeEncryptionKey(base64Key);
```

■ `SetRuntimeEncryptionKey()` overwrites the design-time key only in memory. It does not persist to disk and does not modify the StasisSettings asset.

Key Rotation

This product includes a dedicated Key Rotation service that decrypts all existing save files with the old key and re-encrypts them with the new key, atomically, with backups before each file is overwritten. Access it via Window → RangaDigital → Stasis → Key Rotation, or call the API directly:

C# — Programmatic key rotation

```
using RangaDigital.Stasis.Editor.FileManagement;

// Rotate keys across all save files in the save directory
await StasisKeyRotationService.RotateAllFilesAsync(
    saveDirectory : Application.persistentDataPath,
    oldKey         : oldBase64Key,
    newKey         : newBase64Key
);
```

3.3 Compression

Payload compression is applied after serialisation and before encryption. Three codecs are supported:

LZ4 — Extremely fast compression and decompression with moderate compression ratios. Ideal for games with frequent auto-saves or large save files. Enabled by the `STASIS_COMPRESSION_LZ4` scripting define (set automatically by the Setup window).

GZip — Higher compression ratio at the cost of slower compression speed. Appropriate for platforms where storage is severely constrained (e.g. some console SKUs) and save frequency is low. No additional native dependency required.

Zstd — Best-in-class compression ratio with fast decompression. Recommended for games with very large save files or poor network bandwidth (cloud saves). Requires the ZstdSharp native plugin and the STASIS_COMPRESSION_ZSTD define.

To enable compression, check Compression Enabled in StasisSettings and select the desired codec. The Minimum Payload Size field (default: 512 bytes) prevents compression from being applied to small payloads where the compression overhead exceeds the size saving.

3.4 Auto-Save

This product includes a configurable auto-save subsystem that periodically saves the game without player intervention. Auto-save is disabled by default.

Configuring Auto-Save

Auto Save Enabled	Master switch. Must be checked for any auto-save behaviour to occur.
Auto Save Interval	Time in seconds between auto-save triggers. Minimum: 10 seconds.
Auto Save Slot	The slot name to write auto-saves to. Defaults to __stasis.autosave.
Auto Save On Scene Load	If checked, a save is triggered automatically each time a new scene finishes loading.
Auto Save On Pause	If checked, a save is triggered when the application loses focus or is paused.

To respond to auto-save events, subscribe to StasisManager.OnSaveBegan and StasisManager.OnSaveCompleted:

C# — Respond to auto-save completion

```
StasisManager.Instance.OnSaveCompleted += result =>
{
    if (result.IsAutoSave)
        HudController.ShowToast("Game saved automatically.");
};
```

3.5 Save Slots

Save slots are named identifiers for save files. This product imposes no hard limit on the number of slots. A slot corresponds to one .stasis file on disk plus an optional .stasis.meta sidecar containing slot metadata (timestamp, game version, screenshot thumbnail).

Listing All Slots

C# — Enumerate save slots

```
IReadOnlyList<SaveSlotInfo> slots =
    await StasisManager.Instance.ListSlotsAsync();

foreach (var slot in slots)
{
    Debug.Log($"Slot: {slot.SlotName} Saved: {slot.SavedAt} " +
        $"Version: {slot.GameVersion}");
}
```

Deleting a Slot

C# — Delete a save slot

```
await StasisManager.Instance.DeleteSlotAsync("SaveSlot_1");
```

Screenshot Thumbnails

When Screenshot Capture is enabled in StasisSettings, Stasis captures a thumbnail of the screen at save time and stores it in the .stasis.meta sidecar. Retrieve it for UI display:

C# — Load slot thumbnail

```
Texture2D thumb = await StasisManager.Instance.LoadSlotThumbnailAsync("SaveSlot_0");  
slotImage.texture = thumb;
```

3.6 Atomic Writes and Backup Generations

Every write performed by this product is atomic. The sequence is:

1. Serialise and optionally compress/encrypt the document into a byte array in memory.
2. Write the byte array to a temporary file (.stasis.tmp) in the same directory as the target.
3. If a previous save exists, rename it to the backup path (.stasis.bak).
4. Rename the temporary file to the final path (.stasis). This rename is the single atomic operation.
5. Optionally rotate older backups to .bak.1, .bak.2, etc., up to the configured Max Backup Generations count.

This sequence guarantees that if the process is killed at any point, either the previous save or the new save will be intact. A partially-written .tmp file is never promoted to the primary slot.

■ On Windows, the rename operation (File.Move) is not guaranteed to be atomic at the OS level across different drives. For maximum reliability, ensure Application.persistentDataPath resides on a single drive. Stasis defaults to this configuration.

3.7 Save Data Migration

As the game evolves, the schema of save data changes. Stasis provides a structured migration pipeline that upgrades save files from older game versions to the current schema transparently at load time.

Defining a Migration

C# — Define a migration handler

```
using RangaDigital.Stasis;

// Register this class as a migration handler for version "1.0.0" -> "1.1.0"
[StasisMigration(fromVersion: "1.0.0", toVersion: "1.1.0")]
public class PlayerMigration_1_0_to_1_1 : IStasisMigrationHandler
{
    public void Migrate(SaveDocument document)
    {
        var player = document.GetOrCreateSection("player");

        // Field was renamed from "hp" to "health" in 1.1.0
        if (player.HasField("hp"))
        {
            float hp = player.ReadFloat("hp");
            player.WriteFloat("health", hp);
            player.RemoveField("hp");
        }

        // New field "stamina" was added in 1.1.0 — provide a default
        if (!player.HasField("stamina"))
            player.WriteFloat("stamina", 100f);
    }
}
```

Migration handlers are discovered automatically by reflection. No registration step is required. Stasis applies all applicable migrations in version order when a save file's recorded game version differs from `Application.version`.

3.8 Spawned Object Persistence

Spawned objects are `GameObjects` that did not exist in the original scene — they were instantiated at runtime (enemies, dropped items, destructible terrain pieces). Stasis tracks these objects in a `SpawnedObjects` list within the `SaveDocument` and re-instantiates them at load time.

C# — Register a spawned object for persistence

```
// Record a spawned object before saving
StasisManager.Instance.RegisterSpawnedObject(
    instance      : droppedItem,
    prefabAddress : "Prefabs/DropItem_HealthPotion",
    partitionKey  : "world"
);

// At load time, Stasis will instantiate the prefab at the saved
// position and rotation and call IStasisSaveable.StasisLoad()
// on any registered components.
```

3.9 Chunked Storage (BinaryFast)

When the `BinaryFast` serialiser profile is active, save data is organised into named chunks, each with its own entry in a table of contents at the start of the file. This allows Stasis to deserialise only the chunks required for the current operation, dramatically reducing load times for large save files.

Chunks are defined by assigning a partition key to `SaveSections`. Sections with the same partition key are written to the same chunk:

C# — Assign sections to chunks

```
// Player data – loaded every time
var player = doc.GetOrCreateSection("player", partitionKey: "core");
// NPC state – only loaded when entering world map
var npcs = doc.GetOrCreateSection("npcs", partitionKey: "world");
// Achievement data – loaded once at startup
var achievements = doc.GetOrCreateSection("achievements", partitionKey: "meta");
```

3.10 Platform Profiles

Platform Profiles allow different Stasis configurations to be applied automatically depending on the target platform at runtime. Each profile overrides a subset of the base StasisSettings values.

To add a Platform Profile, open StasisSettings in the Inspector, expand the Platform Profiles section, click Add Profile, and configure the overrides for the target platform (e.g. WebGL, iOS, Android). At runtime, Stasis detects Application.platform and activates the matching profile automatically.

C# — Force platform profile activation

```
// Profiles are applied automatically. To force a specific profile at runtime:
StasisManager.Instance.Settings.ActivatePlatformProfile(RuntimePlatform.WebGLPlayer);
```

3.11 Build Validator

This product includes an automatic pre-build validator (IPreprocessBuildWithReport) that runs before every player build and catches configuration mistakes that would cause problems in production:

DevelopmentJsonOverride in Release Build

Hard error. A Release build with DevelopmentJsonOverride enabled would ship JSON save files to end-users, breaking performance expectations and potentially exposing readable save data.

AES-GCM with Mono Scripting Backend

Hard error in Release builds; warning in Development builds. AES-GCM is unavailable in the Mono runtime. The build would silently fall back to AES-CBC without this check.

BinaryFast Without Link XML

Warning. If the BinaryFast profile is active for an IL2CPP build but the generated link.xml is absent or incomplete, the IL2CPP code stripper will remove MemoryPack formatter types at build time, causing crashes at runtime.

3.12 Optional Integrations

Cinemachine (STASIS_ENABLE_CINEMACHINE)

Saves and restores the state of CinemachineCamera and CinemachineBrain components, including lens settings, priority, enabled state, and world-up override. Add the StasisCinemachineCameraSaveable component to the root of your Cinemachine rig.

Timeline (STASIS_ENABLE_TIMELINE)

Saves and restores the playback state of PlayableDirector components, including time position and play state. Add StasisTimelineSaveable to any GameObject with a PlayableDirector component.

Visual Effect Graph (STASIS_ENABLE_VFX)

Saves and restores the state of VisualEffect components. Useful for persistent environmental effects (e.g. a perpetually-burning torch).

Visual Scripting (STASIS_ENABLE_VISUAL_SCRIPTING)

Exposes Stasis Save and Load as nodes in the Unity Visual Scripting graph editor.

To enable an integration, open Window → RangaDigital → Stasis → Setup and click Sync Optional System Build Defines, or manually add the relevant define to the project's Scripting Define Symbols (Project Settings → Player → Other Settings → Scripting Define Symbols).

4. Technical Guide and Architecture

4.1 System Requirements

Unity Version	6000.0.x (Unity 6) or later. Unity 2022.3 LTS supported with AES-GCM limitation.
Target Platforms	PC (Windows, macOS, Linux), iOS, Android, WebGL, consoles (PS5, Xbox Series, Nintendo Switch via platform-specific builds). All platforms Unity supports.
Scripting Backend	Mono and IL2CPP both supported. AES-GCM requires IL2CPP.
API Compatibility	.NET Standard 2.1 only. .NET Framework is not supported.
Minimum RAM	No additional runtime RAM requirement beyond normal Unity usage. Stasis pools byte arrays to avoid per-save GC allocation.
Disk Space	The Stasis package itself is approximately 2.4 MB. Save file sizes depend entirely on the game's data volume, serialiser profile, and compression setting.
Threading	Stasis uses Task-based async operations. The Unity main thread is never blocked. No dedicated background thread is required; async continuations run on Unity's synchronisation context.

4.2 Architecture Overview

This product is organised into six layers, each with a clearly-defined responsibility and a clean interface to the layer above and below it:

Layer 6 — Application API

StasisManager singleton. Exposes SaveAsync(), LoadAsync(), ListSlotsAsync(), DeleteSlotAsync(), and the event system. This is the only layer the game code interacts with.

Layer 5 — Document Model

SaveDocument, SaveSection, and SpawnedObjectRecord. The in-memory representation of all game state. Completely serialiser-agnostic — sections are written with typed Write* methods and read with typed Read* methods.

Layer 4 — Serialisation

ISaveSerializer implementations: JsonSaveSerializer, BinarySaveSerializer, FastBinarySaveSerializer, and MemoryPackSaveSerializer. Each serialiser converts a SaveDocument to and from a byte array independently of all other concerns.

Layer 3 — Envelope

SaveSerializerEnvelope. Wraps the serialised payload with a binary header containing magic bytes (STAS), format version, a JSON metadata block (slot name, game version, timestamps, flags), and an optional HMAC-SHA256 signature. All formats share this envelope; it is format-agnostic.

Layer 2 — Transforms

CompressionLayer and AesEncryptionLayer. Applied to the payload bytes after serialisation and before writing. Stateless functions: Compress/Decompress, Encrypt/Decrypt. The flags byte in the envelope header records which transforms were applied so that the inverse can be applied on load.

Layer 1 — Storage

ISlotStorage implementations: LocalFileStorage (file system, default), WebGIPlayerPrefsStorage (PlayerPrefs, WebGL fallback). Responsible for atomic writes, backup generation, and platform-specific I/O.

4.3 Binary File Format

The on-disk format for all Stasis save files is the Stasis Envelope (version 4). The format is fully self-describing: every byte required to interpret the file is present within it.

Stasis Envelope Format v4 — Binary Layout

Offset	Size	Field
0	4 bytes	Magic bytes: 0x53 0x54 0x41 0x53 ("STAS")
4	1 byte	Format version (current: 0x04)
5	4 bytes	Header JSON length (uint32, little-endian)
9	N bytes	Header JSON (UTF-8, see schema below)
9+N	8 bytes	Payload length (int64, little-endian)
9+N+8	M bytes	Payload bytes (serialised, optionally compressed/encrypted)
9+N+8+M	4 bytes	Signature length (uint32); 0 if unsigned
9+N+8+M+4	S bytes	Signature (HMAC-SHA256 over header+payload; omitted if length=0)

Header JSON schema (all fields present):

```
{
  "SlotName"       : "SaveSlot_0",
  "GameVersion"    : "1.2.3",
  "StasisVersion"  : "1.0.0",
  "SavedAtUtcTicks": 638412345678901234,
  "TotalPlaytimeSeconds": 3612.4,
  "ByteOrder"      : "LE",
  "Flags"          : 5,
  "IsSigned"       : false,
  "SignatureAlgorithm" : ""
}
```

Flags byte (bitfield):

Bit 0 (0x01)	FlagEncrypted	– payload is encrypted
Bit 1 (0x02)	FlagCompressed	– payload is compressed
Bit 2 (0x04)	FlagBinary	– payload is binary (vs. JSON text)
Bit 3 (0x08)	FlagRandomIv	– AES-CBC with random IV (legacy)
Bit 4 (0x10)	FlagAesGcm	– AES-GCM authenticated encryption
Bit 5 (0x20)	FlagCompressionLz4	– LZ4 compression codec
Bit 6 (0x40)	FlagCompressionZstd	– Zstandard compression codec
Bit 7 (0x80)	FlagChunked	– BinaryFast chunked layout

4.4 Security Architecture

Security is layered and defence-in-depth. The following controls are applied in order:

Encryption (Layer 1)

AES-256 encrypts the payload. The nonce/IV is randomly-generated per save, ensuring that two saves of identical data produce different ciphertext. The key is 256 bits and must be provided as a Base64-encoded string. Key material is never written to disk by Stasis itself.

Authentication (Layer 2, GCM only)

AES-GCM appends a 128-bit authentication tag to the ciphertext. On decryption, if the tag does not match, decryption is aborted and an exception is thrown. This prevents any bit-flip or substitution attack from producing a valid (but incorrect) plaintext.

Signing (Layer 3, optional)

An HMAC-SHA256 signature over the concatenation of the header bytes and payload bytes is appended to the file. This is independent of encryption and can be used with unencrypted saves to detect tampering or accidental corruption.

Binary Fuzz Hardening

Every binary reader validates length fields before allocating memory. An adversarially crafted save file cannot cause an `OutOfMemoryException`, `IndexOutOfRangeException`, or any other unhandled exception. All malformed inputs produce a typed `StasisDeserializationException`.

Build Validator

The pre-build validator catches configuration mistakes (GCM on Mono, Development override in Release) before they reach production, closing common human-error vulnerability vectors.

4.5 Async I/O Pipeline

The complete save pipeline is fully asynchronous end-to-end. The following describes the execution flow for a typical `SaveAsync()` call:

1. Game code calls `await StasisManager.Instance.SaveAsync(document)`.
2. `StasisManager` raises the `OnSaveBegan` event on the main thread.
3. The document is passed to the active `ISaveSerializer.Serialize()` — a synchronous, CPU-bound operation. If the document is large, this can be offloaded to a thread pool thread via `Task.Run()`. The result is a byte array.
4. `CompressionLayer.Compress()` is called synchronously on the byte array (if compression is enabled). LZ4 and GZip compression is fast enough to be imperceptible for typical save sizes.
5. `AesEncryptionLayer.Encrypt*` is called synchronously on the compressed bytes (if encryption is enabled).
6. `SaveSerializerEnvelope.Write()` wraps the bytes with the header and optional signature.
7. `ISlotStorage.WriteAsync()` is called — an async file I/O operation that runs on the thread pool. The main thread is yielded during the write.
8. On completion, `StasisManager` raises `OnSaveCompleted` on the main thread with the result.

4.6 Advanced Configuration

Custom Type Handlers

For types not natively supported by the `Write*/Read*` methods, implement `IStasisTypeHandler` and register it:

C# — Custom type handler for `UnityEngine.Color`

```
public class ColorHandler : IStasisTypeHandler<Color>
{
    public void Write(SaveSection section, string key, Color value)
    {
        section.WriteFloat(key + ".r", value.r);
        section.WriteFloat(key + ".g", value.g);
        section.WriteFloat(key + ".b", value.b);
        section.WriteFloat(key + ".a", value.a);
    }

    public Color Read(SaveSection section, string key, Color defaultValue)
    {
        return new Color(
            section.ReadFloat(key + ".r", defaultValue.r),
            section.ReadFloat(key + ".g", defaultValue.g),
            section.ReadFloat(key + ".b", defaultValue.b),
            section.ReadFloat(key + ".a", defaultValue.a)
        );
    }
}

// Register before first save/load
StasisManager.Instance.RegisterTypeHandler(new ColorHandler());
```

Tamper Response Policy

The `SaveTamperResponsePolicy` enum controls what happens when a signed save file fails signature verification:

ThrowException	Default. Throws <code>StasisDeserializationException</code> . The game must handle this and offer recovery (load backup, start new game).
LogAndContinue	Logs a warning and proceeds with loading. The tampered data will be used. Not recommended for production.
Ignore	Silently ignores the failure and continues. Only appropriate for internal migration tooling or test harnesses.

5. API Reference Guide

This product is a C# library integrated directly into the Unity Engine — it does not expose an HTTP API. This section documents the complete public C# surface area, organised by class. All types reside in the `RangaDigital.Stasis` namespace unless otherwise stated.

■ All async methods return `Task` or `Task<T>`. Await them on the Unity main thread. All methods are thread-safe with respect to one another unless otherwise noted.

5.1 StasisManager

The central singleton coordinating all save and load operations. Attach to a persistent `GameObject` in the scene. Access via `StasisManager.Instance`.

METHOD `Task SaveAsync(SaveDocument document, CancellationToken ct = default)`

Serialises, compresses, encrypts, and writes the document to disk. Returns when the write completes.

Parameters

`document` (`SaveDocument`) — The document to save. Must have a non-empty `SlotName`.

`ct` (`CancellationToken`) — Optional. Cancel the operation before it begins (I/O in flight cannot be cancelled).

Request / Call Example

```
await StasisManager.Instance.SaveAsync(doc);
```

Response / Return Example

```
// SaveAsync returns Task (no return value)
// Listen to OnSaveCompleted for the result.
```

METHOD `Task LoadAsync(string slotName, CancellationToken ct = default)`

Reads the save file for the given slot, decrypts, decompresses, and deserialises it. Returns the populated `SaveDocument`.

Parameters

`slotName` (`string`) — The slot identifier. Must match the `SlotName` used when saving.

`ct` (`CancellationToken`) — Optional cancellation token.

Request / Call Example

```
var doc = await StasisManager.Instance.LoadAsync("SaveSlot_0");
```

Response / Return Example

```
// doc is a fully-populated SaveDocument
Debug.Log(doc.SlotName); // "SaveSlot_0"
Debug.Log(doc.GameVersion); // e.g. "1.2.3"
```

METHOD `Task<ListSlotMetadata> ListSlotsAsync()`

Returns metadata for every save slot present on disk, sorted by save timestamp descending.

Request / Call Example

```
var slots = await StasisManager.Instance.ListSlotsAsync();
```

Response / Return Example

```
// slots[0].SlotName      -> "SaveSlot_0"
// slots[0].SavedAt      -> DateTime
// slots[0].GameVersion   -> "1.2.3"
// slots[0].HasThumbnail  -> true
```

METHOD Task DeleteSlotAsync(string slotName)

Deletes the save file and all backups for the given slot. Irreversible.

Parameters

slotName ([string](#)) — The slot to delete.

Request / Call Example

```
await StasisManager.Instance.DeleteSlotAsync("SaveSlot_2");
```

Response / Return Example

```
// No return value. OnSaveCompleted is not raised for deletes.
```

METHOD SaveDocument CreateDocument(string slotName)

Creates and returns an empty SaveDocument with the given slot name and the current Application.version.

Parameters

slotName ([string](#)) — The slot name to assign to the document.

Request / Call Example

```
var doc = StasisManager.Instance.CreateDocument("SaveSlot_0");
```

Response / Return Example

```
// doc.SlotName      -> "SaveSlot_0"
// doc.GameVersion   -> Application.version
```

METHOD bool SlotExists(string slotName)

Synchronously checks whether a save file exists for the given slot. Does not read the file.

Parameters

slotName ([string](#)) — The slot to check.

Request / Call Example

```
bool exists = StasisManager.Instance.SlotExists("SaveSlot_0");
```

Response / Return Example

```
// true if the .stasis file is present on disk, false otherwise
```

5.1.1 StasisManager Events

Event	Type	Description
-------	------	-------------

OnSaveBegan	Action	Raised on the main thread immediately before a save begins. Argument: slotName.
OnSaveCompleted	Action	Raised on the main thread when a save completes (success or failure). SaveResult.Success is false on error.
OnLoadBegan	Action	Raised on the main thread immediately before a load begins. Argument: slotName.
OnLoadCompleted	Action	Raised on the main thread when a load completes. LoadResult.Document is null on failure.
OnError	Action	Raised for non-fatal errors (e.g. backup creation failed). Does not prevent the primary operation from completing.
OnRecoveryAttempted	Action	Raised when the automatic backup recovery system attempts to load a backup after a primary load failure.

5.2 SaveDocument

The in-memory representation of a single save file. Contains named sections and a list of spawned objects.

C# — SaveDocument public surface

```
public sealed class SaveDocument
{
    public string SlotName      { get; set; }
    public string GameVersion   { get; set; }
    public string StasisVersion { get; }
    public DateTime SavedAt     { get; set; }
    public float TotalPlaytimeSeconds { get; set; }
    public List<SpawnedObjectRecord> SpawnedObjects { get; }

    public SaveSection GetOrCreateSection(string key, string partitionKey = null);
    public bool        HasSection(string key);
    public bool        RemoveSection(string key);
    public IReadOnlyList<string> GetSectionKeys();
}
```

5.3 SaveSection — Supported Field Types

SaveSection exposes a symmetric Write*/Read* API for all supported value types. Every Read* method accepts a default*Value parameter that is returned if the field does not exist (no exception is thrown).

Method Pair	C# Type	Notes
WriteInt32 / ReadInt32	int	Signed 32-bit integer.
WriteInt64 / ReadInt64	long	Signed 64-bit integer.
WriteFloat / ReadFloat	float	IEEE 754 single-precision float.
WriteDouble / ReadDouble	double	IEEE 754 double-precision float.
WriteBool / ReadBool	bool	Boolean value.
WriteString / ReadString	string	UTF-8 string. Null is round-tripped as null.
WriteVector2 / ReadVector2	Vector2	Two floats (x, y).

WriteVector3 / ReadVector3	Vector3	Three floats (x, y, z).
WriteVector4 / ReadVector4	Vector4	Four floats (x, y, z, w).
WriteQuaternion / ReadQuaternion	Quaternion	Four floats (x, y, z, w).
WriteColor / ReadColor	Color	Four floats (r, g, b, a).
WriteColor32 / ReadColor32	Color32	Four bytes (r, g, b, a).
WriteBytes / ReadBytes	byte[]	Raw byte array. Null-safe.
WriteEnum / ReadEnum	Enum	Stored as the underlying int value.
WriteGuid / ReadGuid	Guid	Stored as a 16-byte array.
WriteDateTime / ReadDateTime	DateTime	Stored as UTC ticks (Int64).
WriteTimeSpan / ReadTimeSpan	TimeSpan	Stored as total ticks (Int64).
WriteMatrix4x4 / ReadMatrix4x4	Matrix4x4	16 floats in column-major order.
WriteRect / ReadRect	Rect	Four floats (x, y, width, height).
WriteInt32Array / ReadInt32Array	int[]	Array of signed 32-bit integers.
WriteFloatArray / ReadFloatArray	float[]	Array of single-precision floats.
WriteStringList / ReadStringList	List	List of UTF-8 strings.

5.4 IStasisSaveable

Implement this interface on any MonoBehaviour whose state should be automatically saved and loaded by Stasis when the containing GameObject is saved or restored.

C# — IStasisSaveable implementation

```
public interface IStasisSaveable
{
    // Called by Stasis during SaveAsync. Write component state to section.
    void StasisSave(SaveSection section);

    // Called by Stasis during LoadAsync. Read component state from section.
    void StasisLoad(SaveSection section);
}

// Example implementation
public class PlayerController : MonoBehaviour, IStasisSaveable
{
    public int Level = 1;
    public float Health = 100f;

    public void StasisSave(SaveSection s)
    {
        s.WriteInt32("level", Level);
        s.WriteFloat("health", Health);
    }

    public void StasisLoad(SaveSection s)
    {
        Level = s.ReadInt32("level", Level);
        Health = s.ReadFloat("health", Health);
    }
}
```

5.5 StasisSettings — Key Properties

Property	Type	Description
SerializerProfile	SaveSerializerProfile	Active serialiser (Json, BinaryStandard, BinaryFast).
EncryptionEnabled	bool	Master switch for payload encryption.
EncryptionMode	EncryptionMode	AuthenticatedAesGcm or LegacyAesCbc.
EncryptionKey	string	Base64-encoded 256-bit key. Do not embed in shipped builds.
CompressionEnabled	bool	Master switch for payload compression.
CompressionMode	SaveCompressionMode	LZ4, GZip, or Zstd.
AutoSaveEnabled	bool	Enable periodic auto-save.
AutoSaveIntervalSeconds	float	Time between auto-save triggers (minimum 10 s).
MaxBackupGenerations	int	Number of .bak generations to keep (0 = disabled).
DevelopmentJsonOverride	bool	Force JSON serialiser in Development builds. Blocked in Release builds by the build validator.
SigningEnabled	bool	Enable HMAC-SHA256 file signing.
TamperResponsePolicy	SaveTamperResponsePolicy	Response to signature mismatch: ThrowException, LogAndContinue, or Ignore.

6. Troubleshooting and FAQ

6.1 Common Errors and Resolutions

StasisDeserializationException: Stasis: File does not have a valid Stasis header.

Cause: The file being loaded is not a Stasis save file, or it has been corrupted entirely (e.g. zero-filled by a failed download).

Resolution:

1. Verify the slot name passed to LoadAsync() matches the slot name used when saving.
2. Check that the file at the expected path has the .stasis extension and is non-zero in size.
3. If a .bak backup exists alongside the file, it may be intact. Call StasisManager.Instance.LoadBackupAsync(slotName) to attempt recovery.
4. If this file was transferred from another machine, confirm it was not corrupted during transfer (check file size and SHA-256 hash).

StasisDeserializationException: Save file is encrypted but no decryption key was provided.

Cause: LoadAsync() was called on an encrypted save file without configuring an encryption key in StasisSettings.

Resolution:

1. Open StasisSettings in the Inspector and verify that Encryption Enabled is checked and the Encryption Key field contains the correct Base64 key.
2. If the key is set at runtime via SetRuntimeEncryptionKey(), ensure it is called before LoadAsync().
3. If you are loading a save file from a previous version of the game that used a different key, use the Key Rotation window to re-encrypt the file with the current key.

BUILD_ERROR_GCM_REQUIRES_IL2CPP: Build blocked due to AES-GCM on Mono backend.

Cause: The build validator detected that AES-GCM (AuthenticatedAesGcm mode) is selected but the scripting backend for the target platform is Mono. AES-GCM is unavailable in the Mono runtime.

Resolution:

1. Option A: Switch the scripting backend to IL2CPP in Project Settings → Player → Other Settings → Scripting Backend.
2. Option B: Change the Encryption Mode in StasisSettings to LegacyAesCbc, which is supported on both Mono and IL2CPP. Note that AES-CBC does not provide authenticated encryption.
3. Option C: Disable encryption entirely if the game does not require save-file protection.

BUILD_ERROR_JSON_OVERRIDE_IN_RELEASE: Build blocked due to DevelopmentJsonOverride.

Cause: The DevelopmentJsonOverride flag is enabled in StasisSettings and you are building a Release (non-Development) player.

Resolution:

1. Open StasisSettings in the Inspector.
 2. Uncheck the DevelopmentJsonOverride checkbox.
 3. Rebuild. The build validator will pass.
-

InvalidOperationException: Stasis: Zstandard compression is selected, but 'STASIS_COMPRESSION_ZSTD' is not enabled.

Cause: The SaveCompressionMode is set to Zstd in StasisSettings, but the STASIS_COMPRESSION_ZSTD scripting define has not been added to the project.

Resolution:

1. Open Window → RangaDigital → Stasis → Setup.
 2. Click Sync Optional System Build Defines. This adds the required defines automatically.
 3. Alternatively, add STASIS_COMPRESSION_ZSTD manually to Project Settings → Player → Other Settings → Scripting Define Symbols.
 4. Ensure that ZstdSharp.dll is present in the Assets/Plugins folder. If it is missing, reinstall this product.
-

OutOfMemoryException during LoadAsync() on a large save file.

Cause: The document is extremely large and the serialiser is allocating a full in-memory copy. This is unusual for typical save data but can occur with large embedded binary blobs (e.g. terrain heightmaps).

Resolution:

1. Switch to the BinaryFast serialiser profile, which supports chunked partial loading so only required sections are deserialised.
 2. Reduce the size of embedded binary data. Large assets (textures, meshes) should not be stored in save files; store only references (asset addresses) instead.
 3. If the issue occurs only on low-RAM devices, set a maximum save size guard in StasisSettings → Advanced → Max Payload Size.
-

NullReferenceException: StasisManager.Instance is null.

Cause: Code is calling StasisManager.Instance before the StasisManager MonoBehaviour has executed its Awake() method, or the StasisManager GameObject does not exist in the scene.

Resolution:

1. Ensure a StasisManager component is attached to a GameObject in a scene that loads before any save/load calls are made.
 2. If save/load is required during Awake() of another MonoBehaviour, set Script Execution Order (Project Settings → Script Execution Order) so that StasisManager runs before the calling script.
 3. Do not call StasisManager.Instance from static constructors or field initialisers.
-

MEMORYPACK_LINK_XML_INCOMPLETE: IL2CPP build failed with missing formatter types.

Cause: The BinaryFast (MemoryPack) serialiser profile is active, the build target is IL2CPP, but the generated link.xml file is absent or missing the required MemoryPack assembly preservation entries. The IL2CPP code stripper removed the formatter types.

Resolution:

1. Open Window → RangaDigital → Stasis → Setup.

2. Click Generate Link XML. The file will be created at
Assets/RangaDigital/Stasis/Generated/StasisGenerated.link.xml.
 3. Rebuild the player. The build validator will confirm the link.xml is correct before the IL2CPP strip phase.
 4. If the problem persists, manually verify the link.xml contains entries for MemoryPack.Core, System.Collections.Immutable, and System.Runtime.CompilerServices.Unsafe.
-

6.2 Frequently Asked Questions

Q: Can save files be transferred between platforms (e.g. PC → mobile)?

A: Yes. The Stasis binary format is endian-independent (always little-endian) and platform-agnostic. A save file produced on Windows can be loaded on iOS, Android, or any other Unity-supported platform without modification. Cloud save synchronisation is transparent: copy the .stasis file, and Stasis will load it correctly.

Q: What happens if the player quits mid-save?

A: Nothing is lost. The atomic write sequence ensures that either the previous save or the new save is present on disk — never a partially-written file. The .tmp file is orphaned and will be cleaned up on the next startup (Stasis scans for and deletes orphaned .tmp files in CleanupAsync(), which is called automatically from StasisManager.Awake()).

Q: How do I migrate save files when I add new fields to an existing section?

A: New fields added to a section do not break existing save files. If a Read* call is made for a field that does not exist in the loaded document, the defaultValue parameter is returned instead. No migration handler is needed for additive changes. Migration handlers are only required for destructive changes: field renames, type changes, or section restructuring.

Q: Is there a file size limit?

A: There is no file size limit enforced by this product. File sizes are bounded only by available disk space and the address space of the platform. Practical limits depend on the target hardware; for mobile platforms, save files above 50 MB may cause noticeable load times even with LZ4 compression. The BinaryFast chunked format with partial loading is recommended for documents above 10 MB.

Q: Can Stasis be used with Addressables?

A: Yes. Stasis is Addressables-agnostic. When recording a spawned object, pass the Addressable address string as the prefabAddress parameter to RegisterSpawnedObject(). Stasis stores the address string verbatim. At load time, your own code is responsible for instantiating the prefab at the correct address; Stasis calls StasisLoad() on IStasisSaveable components after instantiation.

Q: Can multiple save operations run concurrently?

A: No. StasisManager queues concurrent save and load requests. If SaveAsync() is called while a previous SaveAsync() is in progress, the second call will await the first before beginning. This prevents race conditions on the underlying file. You do not need to implement any locking in your game code.

Q: How are save files handled on WebGL?

A: On WebGL, the file system is unavailable. Stasis automatically detects the WebGL platform and switches to a PlayerPrefs-backed storage implementation that serialises the save document to a Base64-encoded string and stores it under a namespaced key. The API is identical to all other platforms; no code changes are required. Note that PlayerPrefs on WebGL is browser-specific and not persistent across browsers or incognito sessions.

Q: How do I encrypt with a key derived from the player's account credentials?

A: Call `SetRuntimeEncryptionKey(base64Key)` immediately after your account authentication handshake completes and before the first `LoadAsync()` call. Derive the 256-bit key from the player's credentials using a KDF such as HKDF or PBKDF2 in your own security layer and pass the Base64-encoded result to Stasis. Stasis does not perform key derivation; it accepts a pre-derived key only.

Q: What should I do if a player's save file is corrupted and no backup exists?

A: This product cannot reconstruct data that is not present in any save file or backup. Implement a new-game fallback in your game code that is called when `LoadAsync()` throws `StasisDeserializationException` and no backup is available. For cloud-backed saves, offer the player the option to download from the cloud. Enable `MaxBackupGenerations` (at least 2) to reduce the probability of this scenario.

Q: How does this product compare to EasySave3?

A: EasySave3 is the market-leading save system on the Unity Asset Store and is an excellent product for rapid prototyping and small-to-medium projects. This product targets professional productions that require features EasySave3 does not provide: authenticated encryption (AES-GCM), atomic writes, multi-generation backups, a structured migration pipeline, partial-load chunked binary format, and a pre-build validator that catches configuration errors before they ship.
